

CUSTOMOWE ZNACZKI!

Lektura do poduszki: [HD44780.pdf \(sparkfun.com\)](http://HD44780.pdf(sparkfun.com))

HD44780U

Table 4 Correspondence between Character Codes and Character Patterns (ROM Code: A00)

Lower 4 Bits	Upper 4 Bits	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
xxxx0000	CG RAM (1)			0	@	P	`	P				-	9	≡	α	p	
	(2)		!	1	A	Q	a	9			。	ア	チ	△	ä	q	
xxxx0010	(3)		"	2	B	R	b	r			「	イ	ツ	×	β	θ	
	(4)		#	3	C	S	c	s			」	ウ	テ	モ	ε	∞	
xxxx0100	(5)		\$	4	D	T	d	t			、	エ	ト	ト	μ	Ω	
	(6)		%	5	E	U	e	u			・	オ	ナ	ユ	ε	Ü	
xxxx0110	(7)		&	6	F	V	f	v			ヲ	カ	ニ	ヨ	ρ	Σ	
	(8)		'	7	G	W	g	w			ア	キ	ヌ	ラ	g	π	
xxxx1000	(1)		(	8	H	X	h	x			ィ	ク	ネ	リ	フ	×	
	(2)		)	9	I	Y	i	y			ウ	ケ	ノ	ル	リ	ユ	
xxxx1010	(3)		*	:	J	Z	j	z			エ	コ	ハ	レ	j	≠	
	(4)		+	;	K	[	k	{			オ	サ	ヒ	ロ	*	π	
xxxx1100	(5)		,	<	L	¥	l				ヤ	シ	フ	ワ	Φ	円	
	(6)		-	=	M	]	m	}			ユ	ズ	ハ	ン	も	÷	
xxxx1110	(7)		.	>	N	^	n	→			ヨ	セ	ホ	°	°		
	(8)		/	?	O	_	o	+			ッ	リ	マ	°	ö	■	

Note: The user can specify any pattern for character-generator RAM.

```

#include "xc.h"
#include <libpic30.h>

// Definicja makr tak by kod był czytelny, przejrzysty, deskryptywny i
przyjazny użytkownikowi
#define FCY      4000000UL    // częstotliwość robocza oscylatora jako połowa
częstotliwości (FNOSC = FRC -> FCY = 4000000)
// Zdefiniowanie poszczególnych pingów jako odpowiednie makra
#define LCD_E      LATDbits.LATD4
#define LCD_RW      LATDbits.LATD5
#define LCD_RS      LATBbits.LATB15
#define LCD_DATA    LATE

// Przypisanie wartości poszczególnych komend do właściwych makr
#define LCD_CLEAR    0x01      //0b00000001
#define LCD_HOME     0x02      //0b00000010
#define LCD_ON       0x0C      //0b00001100
#define LCD_OFF      0x08      //0b00001000
#define LCD_CONFIG    0x38      //0b00111000
#define LCD_CURSOR    0x80      //0b10000000
#define LINE1         0x00
#define LINE2         0x40
#define LCD_CUST_CHAR  0x40      //0b01000000
#define LCD_SHIFT_R    0x1D      //0b00011100
#define LCD_SHIFT_L    0x1B      //0b00011000

// Definicja funkcji delay w us i ms - operujących na jednostkach czasu
zamiast cykli pracy oscylatora

void __delay_us(unsigned long us){
    __delay32(us*FCY/1000000);
}

void __delay_ms(unsigned long ms){
    __delay32(ms*FCY/1000);
}

// Definicja funkcji wysyłających komendy (RS = 0) i dane (RS = 1) za
pomocą magistrali równoległej (LCD_DATA). Znaki i komendy mają 8 bitów!

void LCD_sendCommand(unsigned char command){
    LCD_RW = 0;    // Zapis
    LCD_RS = 0;    // Przesyłanie komend
    LCD_E = 1;     // Otwarcie transmisji danych
    LCD_DATA = command;
    __delay_us(50);    // Opóźnienie konieczne dla zapisania danych.
    LCD_E = 0;       // Konieczne zablokowanie transmisji po przesłaniu
    komunikatu.
}

```

```

void LCD_sendData(unsigned char data){
    LCD_RW = 0;
    LCD_RS = 1;    // Przesyłanie danych
    LCD_E = 1;
    LCD_DATA = data;
    __delay_us(50);
    LCD_E = 0;
}

// Funkcja print wyświetlająca kolejne 8-bitowe znaki w pętli while, *
// oznacza przypisanie nie wartości zmiennej lecz jej adresu.

void LCD_print(unsigned char* string){
    while(*string){
        LCD_sendData(*string++);
    }
}

// Funkcja ustawiająca kursor w wybranym miejscu

void LCD_setCursor(unsigned char row, unsigned char col){
    unsigned char address;
    if (row == 1){
        address = LCD_CURSOR + LINE1 + col;
    }
    if (row == 2){
        address = LCD_CURSOR + LINE2 + col;
    }
    LCD_sendCommand(address);
}

// Funkcja ZAPISUJĄCA znak (zmienna array) do PAMIĘCI CGRAM na wybranym
// słocie od 0 do 7

void LCD_saveCustChar(unsigned char slot, unsigned char *array) {
    unsigned char i;
    LCD_sendCommand(LCD_CUST_CHAR + (slot*8));
    for(i=0;i<8;i++){
        LCD_sendData(array[i]);
    }
}

// Wybór znaku do wyświetlenia z tablicy

void LCD_displayCharacter(unsigned char characterCode) {
    LCD_sendData(characterCode);
}

```

```
// Funkcja inicjalizująca wyświetlacz LCD. Wysyła niezbędne komendy jak  
LCD_CONFIG i LCD_ON
```

```
void LCD_init(){  
    __delay_ms(20);  
    LCD_sendCommand(LCD_CONFIG);  
    __delay_us(50);    // opóźnienia wynikają ze specyfikacji  
wyświetlacza i czasu przetwarzania poszczególnych komend  
    LCD_sendCommand(LCD_ON);  
    __delay_us(50);  
    LCD_sendCommand(LCD_CLEAR);  
    __delay_ms(2);  
}
```

```
// Zdefiniowanie znaku niestandardowego w postaci tablicy 8x5 (8 linii po  
5 kropek)
```

```
unsigned char symbol1[8] = {  
    0b00100,  
    0b01010,  
    0b10001,  
    0b10001,  
    0b10001,  
    0b01010,  
    0b00100,  
    0b00000  
};
```

```
unsigned char symbol2[8] = {  
    0b11011,  
    0b10101,  
    0b01110,  
    0b01110,  
    0b01110,  
    0b10101,  
    0b11011,  
    0b00000  
};
```

```
int main(void) {  
    TRISB = 0x7FFF;    // Ustawienie rejestrów kierunku  
    TRISD = 0x0000;  
    TRISE = 0x0000;  
  
    LCD_init();    // Inicjalizacja wyświetlacza  
    LCD_saveCustChar(0, symbol1); // Zapisanie znaku 'symbol1' do pamięci  
CGRAM  
    LCD_saveCustChar(1, symbol2); // Zapisanie znaku 'symbol2' do pamięci  
CGRAM
```

```

while(1){
    LCD_setCursor(2,0); // Ustawienie kursora na początku drugiej
linii
    LCD_print("Hello world!"); // Wyświetlenie napisu
    LCD_sendData(0); // Wyświetlenie znaku ze slotu 0 w pamięci
CGRAM
    LCD_sendData(1); // Wyświetlenie znaku ze slotu 1 w pamięci
CGRAM
    LCD_displayCharacter(11010000);
    __delay_ms(1000);
    LCD_sendCommand(LCD_CLEAR);
    __delay_ms(500);
}
return 0;
}

```